+1



THE FOUR CORRECTIONS

4

four values that inherited authority they had not earned, and what happened when each was recomputed

peterlodri · the constellation

{n+−1−<△>} · 0+1 · fine touch from within · vaked.dev

I

THE TRIT THAT WAS TYPED

A manifest printed three twelve-trit registers and a collapsed state `Σ(t) mod 3`. Three principals, twelve columns, one claim: add the columns, reduce mod three, and this is what comes out.

The rule is well formed, so the arithmetic is checkable, so it was checked. Ten of twelve columns reproduce. Two do not. And the two that fail are the same computation reported two different ways: both columns sum to `2`, and one is printed as `+` while the other is printed as `0`. Under any single rule those two must come out identical, because their inputs are identical. Neither is what the rule yields. A rule cannot map the same input to two different outputs; a hand can.

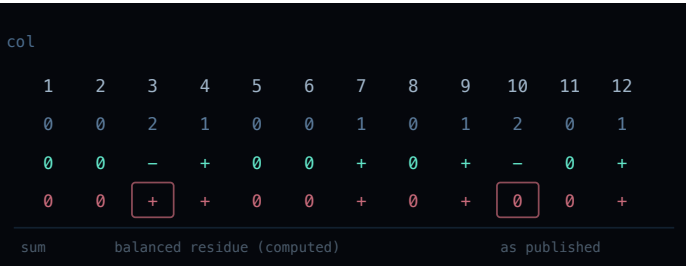


fig 1 · ten columns agree, two do not, and the two disagree with each other

The correction: the computed collapse is `0 0 − + 0 0 + 0 + − 0 +`. It was published beside the original rather than over it, because the original is the evidence that the manifest was assembled by hand in the spirit of a rule rather than run through it.

II

THE SEAL THAT DECODED

The same page carried another seal. Ten bytes of hex, printed under a heading that promised nothing in particular:

```
0x54 0x52 0x49 0x4E 0x49 0x54 0x59 0x2D 0x38 0x42
```

Decoded, that is `TRINITY-8B`, exact to the byte. So the two seals on the page were not equally honest, and the difference was not visible in the typography: both were monospace, both were confident, both sat in the same table. One had been computed. The other had been written by feel.

> *The seal that could be computed was computed correctly. The seal that required arithmetic was written by feel. Both were published with equal confidence.*

The correction: none was needed for the hex. The correction was to the reader's assumption — that a page

which gets one seal exactly right has probably got the other one right too. It has not, and the assumption is the bug, not the arithmetic.

III

THE DIGEST THAT WALKED

A receipt for a compressed container printed two SHA-256 digests. A copy of that receipt, paraphrased, carried a different pair — and the same value twice, for two different byte strings, which is impossible for a real pair.

The tell was a stride. Step through the pasted bytes and count the differences:

string	steps equal to +0x22
the pasted value	12 of 31
the real file digest	0 of 31

A hash output is indistinguishable from noise, so a given stride appears about once in 256 steps by chance. Twelve is not chance. *A hash that walks in step was typed, not computed.*

The correction: the artifact was innocent. The container verified nine ways — magic, block count, raw size, the XOR

fold, the payload digest, the file digest, the savings — and the container additionally stores the digest of its own decompressed payload, so the file proves itself and the receipt only quotes it. What failed was the *copy*: a regeneration of a receipt from a description of a receipt.

IV

THE MEASUREMENT THAT WAS TRUE

The first three corrections are the same species: a value that was never computed, dressed as one that was. The fourth is gentler and more common.

A book about a training run states its parity tolerance, then reports the measurement underneath it:

```
Not approximately. 1e-5.
prompt 1: max_abs 9.584e-05
prompt 2: max_abs 9.918e-05
Bit for bit.
```

`9.584e-05` is 9.6 times larger than `1e-5`. `9.918e-05` is 9.9 times larger. And a nonzero `max_abs` is by definition the measurement of a difference, so it is not what *bit for bit* describes.

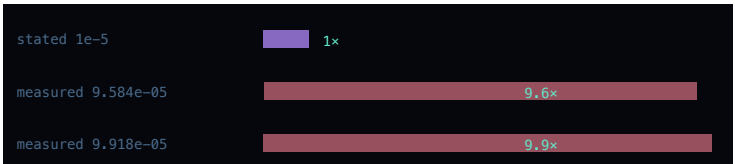


fig 2 · the tolerance was the label; the measurement was the truth

What holds is the operational claim. Both prompts keep the same argmax, so the deployed body chooses what the trained mind chose. That is *argmax-for-argmax* parity at ~1e−4, which is a real result. The book oversold the label, not the work.

The correction: a note under the original paragraph, keeping both the sentence and the measurement, and naming which one was wrong.

V

WHAT THE FOUR TEACH

Three of these were **provenance** failures: a value that was never computed inherited the authority of values that were. One was a **labelling** failure: a true number under a false word.

Both are the same structural error, and it is the error the whole shelf is built against. A projection is substituted for the thing projected, and then the projection inherits the authority of the original. Compression does not preserve admissibility. A summary keeps the conclusion and drops the provenance. The conclusion then behaves like something that was measured.

The defences are unglamorous, and each one is a sentence you can run:

defence	form
recompute, do not reprint	`verify-zkp1.py <container> --expect-seal …`

make the rule a type	`python3 -m vakedc check --json …`
keep the correction in place	the original paragraph, and the note under it
label the ladder	`named` < `verified`, in the file, not the prose

And the rule that makes the rest usable:

> *A plausible branch may be ranked; only a verified branch may be bound. Replayable ⇒ admissible.*

None of the four corrections deletes anything. The trit table still shows the published collapse next to the computed one. The Tokyo page still carries both seals. The OMNI receipt now carries the command that re-derives it. The eclipse book keeps the sentence that was wrong about its own tolerance, with the recomputation directly beneath it.

That is the whole method, and it is a filing system rather than a virtue: keep the fracture visible, fill it, and make the next reader's check cheaper than your claim. A vault that treated the zero as nothing would have to hide its seams. This one files them.

Four values, four checks, one sentence that survived all of them:

```
{<3,<3,<3}+1
```

pocoo sovereign library · 0 + 1 · fine touch from within · vaked.dev